Tugas Besar 1 : Searching

IF3170 Inteligensi Artifisial 2026/2027

FrontierFolly



Anggota
Mohammad Najmutstsaqib 13223040
Ega Luthfi Rais 13524115
Ahmad Fauzan Putra 13524141

I. Deskripsi Persoalan

Tugas ini bertujuan untuk membuat konfigurasi pengisian paket paling optimal untuk sebuah truk peti kemas. Optimal disini berarti **memaksimalkan** fungsi objektif.

Daftar Variabel Permasalahan

Entitas	Atribut	Contoh	Keterangan
Package	id	"RusdiPackage"	*Identifier*
	Dimensions	{w:3, l:4, h:1}	Ukuran sebuah paket, yaitu *width*, *length*, dan *height*
	Orientation	{w:x, l:y, h:z}	Sumbu yang sejajar dengan setiap sisi paket. Terdapat enam kemungkinan orientasi, misalnya {w:y, l:x, h:z} setelah rotasi 90° terhadap sumbu *z*.
	Value	60	Nilai harga sebuah paket (satuan unit)
	Weight	5	Berat paket (satuan unit berat)
	IsFragile	False	Benda pecah-belah
	ETA	3	Estimated Time of Arrival, perkiraan waktu kedatangan paket (dalam sisa hari)
Truck	Dimensions	{w:10, l:20, h:5}	Ukuran sebuah peti kemas pada truk, yaitu *width*, *length*, dan *height*
	MaxCapacity	500	Kapasitas angkut maksimum pada suatu peti kemas (satuan unit berat)

Ketentuan Representasi

Berikut prosedur bagaimana dalam merepresentasikan permasalahan ini:

- Paket hanya berbentuk kubus / balok. Paket dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam peti kemas pada truk sehingga suatu paket dapat berada di luar **(seluruh bagian paket di luar)**
- Koordinat berupa bilangan bulat. Cara representasi koordinat dibebaskan.
- Struktur data yang digunakan untuk representasi dibebaskan. Misalnya menggunakan *key-value* seperti JSON.
- Inisialisasi (***initial state***) random.
- Langkah neighbor (***move***) yang diperbolehkan tiap iterasi:

1. Menukar dua paket, baik di dalam maupun di luar truk
2. Memindahkan sebuah paket ke suatu koordinat yang berbeda
3. Merotasi paket 90° terhadap sumbu *x*, *y*, atau *z*

- Representasi solusi (***final state***) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada truk. Secara detail yang harus dapat ditunjukkan oleh state dan visualisasi adalah:

    1. Bagaimana keadaan semua paket yang di dalam maupun yang di luar
    2. Bagaimana keadaan peti kemas pada truk
    3. *Value* pada *state* tersebut

Objective Function & Constraints

Agar permasalahan ini tidak menjadi kompleks, berikut arahan yang diberikan

- **Objective function**
  Permasalahan ini bertujuan untuk memaksimalkan *value* peti kemas pada truk.
- **Kardus tidak bisa melayang dan menembus dinding peti kemas**
  Melayang disini didefinisikan dengan misalkan paket i setelah orientasi tertentu memiliki posisi sudut kiri-bawah-belakang

  $$(x_i, y_i, z_i)$$

  dan dimensi

  $$(w_i, d_i, h_i)$$

  sehingga alas paket i adalah

  $$B_i = \{(x, y, z_i) \mid x_i \leq x < x_i + w_i,\ y_i \leq y < y_i + d_i\})$$

  Sebuah titik di alas memiliki *support* apabila:

  1. $Z_{i=0}$, artinya di alas truk, atau
  2. Terdapat paket lain j yang mempunyai bagian atas tepat di bawah titik tersebut

  Maka melayang adalah kondisi tidak ada satupun titik alas yang memiliki support.

- **Barang pecah-belah tidak boleh di bawah paket lain**

  Barang pecah-belah tidak boleh menjadi *support*

- **Truk tidak boleh mengangkut melebihi kapasitasnya**

II. Pembahasan

Representasi State dan Fungsi Objektif

State adalah *dict* yang memetakan id paket ke posisi dan orientasi paket. Orientasi adalah sumbu-sumbu paket dengan 0 sebagai x dan seterusnya.

```
{
  "Z": {"position": [3, 14, 0], "orientation": [0, 1, 2]},
  "K": {"position": null,       "orientation": [2, 1, 0]}
}
```

Fungsi objektif adalah total value paket di dalam truk.

```
def packageScore(package, objective="value"):
    if objective == "value":
        return package["value"]
    if objective == "urgency":
        # Bonus
        return package["value"] / package["eta"]
    raise ValueError(f"Objective '{objective}' tidak dikenali")

def stateScore(state, packages, objective="value"):
    return sum(
        packageScore(packages[k], objective)
        for k, placement in state.items()
        if placement["position"] is not None
    )
```

Constraint

state_valid memeriksa enam constraint: orientasi, tidak tembus dinding, tidak tabrakan, total berat tidak di atas kapasitas, tidak melayang, dan paket fragile tidak di tumpangi paket lain.

```
# Apakah state aman dari constraint
def state_valid(state, packages, truck):
    return (check_orientation(state)
            and check_bounds(state, packages, truck)
            and check_collision(state, packages)
            and check_weight(state, packages, truck)
            and check_support(state, packages)
```

```
            and check_fragile(state, packages))
```

contact *true* bila bagian atas paket a menyentuh alas paket b dan keduanya beririsan pada sumbu x dan y.

```
# Cek apakah ada kontak dari package a (penopang) dan package b (yang
ditopang)
def contact(pos_a, pos_b, rotate_dims_a, rotate_dims_b):
    z_contact = (pos_a[2] + rotate_dims_a[2] == pos_b[2])
    x_contact = (
        (pos_a[0] < (pos_b[0] + rotate_dims_b[0])) and
        (pos_b[0] < (pos_a[0] + rotate_dims_a[0]))
    )
    y_contact = (
        (pos_a[1] < (pos_b[1] + rotate_dims_b[1])) and
        (pos_b[1] < (pos_a[1] + rotate_dims_a[1]))
    )

    return z_contact and x_contact and y_contact
```

check_support mencegah paket melayang

```
# Cek apakah tidak ada package yang melayang
def check_support(state, packages):
    list_state_id = list(state.keys())
    state_len = len(state)
    for i in range(state_len):
        package_1 = list_state_id[i]
        pos_1 = state[package_1]["position"]
        if (pos_1 is None) or (pos_1[2] == 0):
            continue

        dims_1 = packages[package_1]["dimensions"]
        ori_1 = state[package_1]["orientation"]

        rotate_dims_1 = [dims_1[k] for k in ori_1]

        supported = False
        for j in range(state_len):
            package_2 = list_state_id[j]
            pos_2 = state[package_2]["position"]
            if (i == j) or (pos_2 is None):
                continue
```

```
            dims_2 = packages[package_2]["dimensions"]
            ori_2 = state[package_2]["orientation"]

            rotate_dims_2 = [dims_2[k] for k in ori_2]

            if contact(pos_2, pos_1, rotate_dims_2, rotate_dims_1):
                supported = True
                break

        if not supported:
            return False

    return True
```

check_fragile menolak state bila ada paket yang berada di atas paket fragile.

```
def check_fragile(state, packages):
    list_state_id = list(state.keys())
    state_len = len(state)
    for i in range(state_len):
        package_1 = list_state_id[i]
        pos_1 = state[package_1]["position"]
        if (pos_1 is None) or (not packages[package_1]["is_fragile"]):
            continue

        dims_1 = packages[package_1]["dimensions"]
        ori_1 = state[package_1]["orientation"]

        rotate_dims_1 = [dims_1[k] for k in ori_1]

        for j in range(state_len):
            package_2 = list_state_id[j]
            pos_2 = state[package_2]["position"]
            if (i == j) or (pos_2 is None):
                continue

            dims_2 = packages[package_2]["dimensions"]
            ori_2 = state[package_2]["orientation"]

            rotate_dims_2 = [dims_2[k] for k in ori_2]
```

```
            if contact(pos_1, pos_2, rotate_dims_1, rotate_dims_2):
                return False

    return True
```

Initial State dan Visualisasi

possible_height mengumpulkan ketinggian z yang bisa di gunakan

```
# Mendapatkan ketinggian yang mungkin bisa ditempati
def possible_height(state, packages):
    height = [0]

    for package_id, placement in state.items():
        pos = placement["position"]

        if ((pos is None) or (packages[package_id]["is_fragile"])):
            continue

        dimensions = packages[package_id]["dimensions"]
        orientation = placement["orientation"]
        rotate_dims = [dimensions[k] for k in orientation]

        top_z = pos[2] + rotate_dims[2]
        height.append(top_z)

    return list(set(height))
```

gen_initial_state mengacak urutan paket, banyak paket, orientasi, dan posisinya.

```
# Generate inisialisasi state secara random
def gen_initial_state(packages, truck):
    state = {}

    for package_id in packages:
        state[package_id] = {
            "position" : None,
            "orientation" : (0, 1, 2)
        }

    list_package_id = list(packages.keys())
    random.shuffle(list_package_id)
```

```
    num_packages = random.randint(1, len(list_package_id))

    truck_dims = truck["dimensions"]

    for package_id in list_package_id[:num_packages]:
        dimensions = packages[package_id]["dimensions"]

        for _ in range(50):
            orientation = tuple(random.sample([0, 1, 2], k=3))
            rotate_dims = [dimensions[i] for i in orientation]

            if any(rotate_dims[k] > truck_dims[k] for k in range(3)):
                continue

            heights = [
                z for z in possible_height(state, packages)
                if z + rotate_dims[2] <= truck_dims[2]
            ]

            if not heights:
                continue

            position = (
                random.randint(0, truck_dims[0] - rotate_dims[0]),
                random.randint(0, truck_dims[1] - rotate_dims[1]),
                random.choice(heights)
            )

            state[package_id] = {
                "position": position,
                "orientation": orientation
            }

            if state_valid(state, packages, truck):
                break

            state[package_id]["position"] = None

    return state
```

plotState menunjukkan paket di dalam truk. plotValues menunjukkan kurva nilai objektif

```
def plotState(state, packages, truck, path):
    W, L, H = truck["dimensions"]
    a = plt.figure(figsize=(4,4)).add_subplot(projection="3d")
    faces, colors = [], []
    palette = plt.cm.tab20.colors + plt.cm.tab20b.colors
    for id, x in state.items():
        if x["position"] is None:
            continue
        (px, py, pz), (w, l, h) = x["position"],
[packages[id]['dimensions'][i] for i in x["orientation"]]
        faces += box(px, py, pz, w, l, h)
        colors += [palette[list(packages).index(id)]] * 6
    a.add_collection3d(Poly3DCollection(faces, facecolor=colors,
edgecolor="blue"))
    a.add_collection3d(Poly3DCollection(box(0, 0, 0, W, L, H), facecolor=(0,
0, 0, 0), edgecolor="black"))
    a.set(xlim=(0, W), ylim=(0, L), zlim=(0, H), xlabel="x", ylabel="y",
zlabel="z")
    a.set_box_aspect((W, L, H), zoom=1)
    a.grid(False)
    for x in (a.xaxis, a.yaxis, a.zaxis):
        x.set_pane_color("white")
    plt.savefig(path, dpi=100)
    plt.close()
```

Steepest Ascent Hill Climbing

Setiap iterasi mengambil *neighbor* dengan peningkatan terbesar, dan berhenti jika tidak ada neighbor yang lebih baik.

axisCandidates membatasi kandidat koordinat pada dinding truk dan sisi paket lain.

```
# Deterministik
def axisCandidates(boxes, axis, size, limit):
    values = {0, limit - size}
    for position, dims, _ in boxes:
        near = position[axis]
        far = near + dims[axis]
        values.update((near, far, near - size, far - size))
    return sorted(v for v in values if 0 <= v <= limit - size)
```

findFeasiblePlacement *return* posisi dan orientasi valid pertama untuk sebuah paket.

```
def findFeasiblePlacement(state, package_id, packages, truck, exclude=None):
    W, L, H = truck["dimensions"]
    fragile = packages[package_id]["is_fragile"]

    if totalWeight(state, packages) + packages[package_id]["weight"] >
truck["max_capacity"]:
        return None

    boxes = insideBoxes(state, packages)
    heights = possible_height(state, packages)

    for orientation in ORIENTATIONS:
        w, l, h = rotatedDims(package_id, orientation, packages)
        if w > W or l > L or h > H:
            continue

        xs = axisCandidates(boxes, 0, w, W)
        ys = axisCandidates(boxes, 1, l, L)
        zs = sorted(z for z in heights if z + h <= H)
        if not xs or not ys or not zs:
            continue

        for z in zs:
            # Overlap z
            rel = [
                (p[0], p[1], p[2], d[0], d[1], d[2], f)
                for p, d, f in boxes
                if (p[2] < z + h and z < p[2] + d[2]) or p[2] + d[2] == z or
p[2] == z + h
            ]

            for x in xs:
                for y in ys:
                    if exclude is not None and (x, y, z) == exclude:
                        continue

                    valid = True
                    supported = z == 0
                    for kx, ky, kz, kw, kl, kh, k_fragile in rel:
                        if not (overlap1D(x, w, kx, kw) and overlap1D(y, l,
ky, kl)):
```

```
                            continue
                        if kz < z + h and z < kz + kh:
                            valid = False
                            break
                        if kz + kh == z:
                            if k_fragile:  # fragile tidak boleh menjadi
penopang
                                valid = False
                                break
                            supported = True
                        elif kz == z + h and fragile:
                            valid = False
                            break

                if valid and supported:
                    return (x, y, z), orientation

    return None
```

bestImprovingMove memilih langkah tambah atau swap dengan kenaikan terbesar.

```
# move
def bestImprovingMove(state, packages, truck, objective="value"):
    scores = {k: packageScore(packages[k], objective) for k in packages}
    inside = [k for k, placement in state.items() if placement["position"] is
not None]
    outside = [k for k, placement in state.items() if placement["position"] is
None]

    best = None

    # highest delta
    for j in sorted(outside, key=lambda k: (-scores[k], k)):
        if scores[j] <= 0:
            break
        placement = findFeasiblePlacement(state, j, packages, truck)
        if placement is None:
            continue
        position, orientation = placement
        new = fotokopi(state)
        new[j] = {"position": position, "orientation": orientation}
        if state_valid(new, packages, truck):
            best = (scores[j], new, f"tambah paket {j} ke {position} orientasi
```

```
{orientation}")
            break

    pairs = [
        (scores[j] - scores[i], i, j)
        for i in inside
        for j in outside
        if scores[j] > scores[i]
    ]
    pairs.sort(key=lambda pair: (-pair[0], pair[1], pair[2]))

    for delta, i, j in pairs:
        if best is not None and delta <= best[0]:
            break
        slot = state[i]["position"]
        reduced = fotokopi(state)
        reduced[i] = {"position": None, "orientation":
state[i]["orientation"]}
        for orientation in ORIENTATIONS:
            new = fotokopi(reduced)
            new[j] = {"position": slot, "orientation": orientation}
            if state_valid(new, packages, truck):
                best = (delta, new, f"swap paket dalam {i} dengan paket luar
{j}")
                break

    return best
```

steepestAscent mengulang sampai maks iterasi

```
def steepestAscent(packages, truck, objective="value", maxIter=2000,
seed=None):
    if seed is not None:
        random.seed(seed)

    start = time.perf_counter()
    initial = gen_initial_state(packages, truck)
    current = fotokopi(initial)
    current_value = stateScore(current, packages, objective)

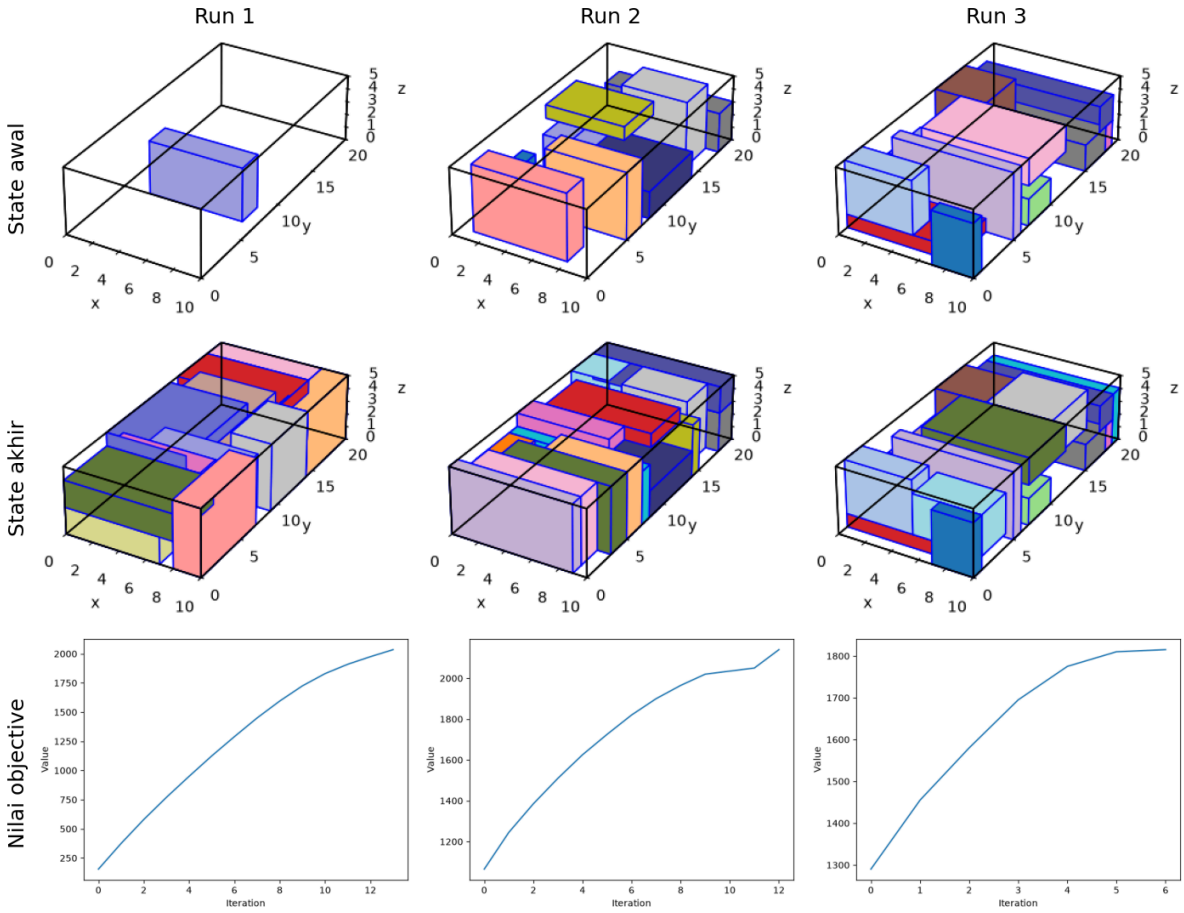
    values = [current_value]
    iterations = 0
```

```
    while iterations < maxIter:
        best = bestImprovingMove(current, packages, truck, objective)
        if best is None:
            break
        _, new_state, _ = best
        current = new_state
        current_value = stateScore(current, packages, objective)
        values.append(current_value)
        iterations += 1

    return {
        "variant": "steepest_ascent",
        "objective": objective,
        "seed": seed,
        "initial": initial,
        "final": current,
        "finalVal": current_value,
        "realValue": rawValue(current, packages),
        "duration": time.perf_counter() - start,
        "valuesOT": values,
        "iterations": iterations,
        "max_iter": maxIter
    }
```

Tabel 1. Hasil steepest ascent hill climbing.

Run (seed)	Nilai akhir	Durasi (detik)	Banyak iterasi
1	2035	0,039	13
2	2140	0,122	12
3	1815	0,136	6



Gambar 1. Steepest ascent hill climbing, run 1 sampai 3.

Pencarian berhenti setelah 6 sampai 13 iterasi dan selesai di bawah 0,15 detik. Nilai akhirnya berbeda jauh antar run (1815–2140) karena local optimum yang dicapai bergantung pada initial state.

Stochastic Hill Climbing

Neighbor dibangkitkan secara *random* dari lima langkah: swap, add, remove, move, dan rotate. Neighbor pertama yang lebih baik langsung diambil. Pencarian berhenti jika 100 percobaan berturut-turut tanpa perubahan.

randomNeighbor memilih satu tipe langkah random dan *return* neighbor yang valid, atau None.

```
def randomNeighbor(state, packages, truck, t=50):
    W, L, H = truck["dimensions"]
    inside = [k for k, placement in state.items() if placement["position"] is
not None]
```

```
    outside = [k for k, placement in state.items() if placement["position"] is
None]
    if not inside and not outside:
        return None

    move = random.choice(["swap", "add", "remove", "move", "rotate"])

    if move == "swap":
        if len(packages) < 2:
            return None
        a, b = random.sample(list(packages), 2)
        if state[a]["position"] is None and state[b]["position"] is None:
            return None
        new = fotokopi(state)
        new[a]["position"], new[b]["position"] = state[b]["position"],
state[a]["position"]
        if state_valid(new, packages, truck):
            return new
        return None

    if move == "add" and outside:
        j = random.choice(outside)
        for _ in range(t):
            orientation = random.choice(ORIENTATIONS)
            w, l, h = rotatedDims(j, orientation, packages)
            if w > W or l > L or h > H:
                continue
            heights = [z for z in possible_height(state, packages) if z + h <=
H]
            if not heights:
                continue
            position = (random.randint(0, W - w), random.randint(0, L - l),
random.choice(heights))
            new = fotokopi(state)
            new[j] = {"position": position, "orientation": orientation}
            if state_valid(new, packages, truck):
                return new
        return None

    if move == "remove" and inside:
        i = random.choice(inside)
        new = fotokopi(state)
        new[i]["position"] = None
```

```
        if state_valid(new, packages, truck):
            return new
        return None

    if move == "move" and inside:
        j = random.choice(inside)
        orientation = state[j]["orientation"]
        w, l, h = rotatedDims(j, orientation, packages)
        reduced = fotokopi(state)
        reduced[j]["position"] = None
        for _ in range(t):
            heights = [z for z in possible_height(reduced, packages) if z + h
<= H]
            if not heights:
                continue
            position = (random.randint(0, W - w), random.randint(0, L - l),
random.choice(heights))
            if position == state[j]["position"]:
                continue
            new = fotokopi(reduced)
            new[j] = {"position": position, "orientation": orientation}
            if state_valid(new, packages, truck):
                return new
        return None

    if move == "rotate" and inside:
        j = random.choice(inside)
        current_ori = state[j]["orientation"]
        others = [o for o in ORIENTATIONS if o != current_ori]
        random.shuffle(others)
        for orientation in others:
            new = fotokopi(state)
            new[j]["orientation"] = orientation
            if state_valid(new, packages, truck):
                return new
        return None

    return None
```

Terlihat stochastic dijalankan dengan batas 100 percobaan per iterasi.

```
def stochastic(packages, truck, objective="value", trials=100, maxIter=2000,
seed=None):
```

```
    if seed is not None:
        random.seed(seed)

    start = time.perf_counter()
    initial = gen_initial_state(packages, truck)
    current = fotokopi(initial)
    current_value = stateScore(current, packages, objective)

    values = [current_value]
    iterations = 0
    total_attempts = 0

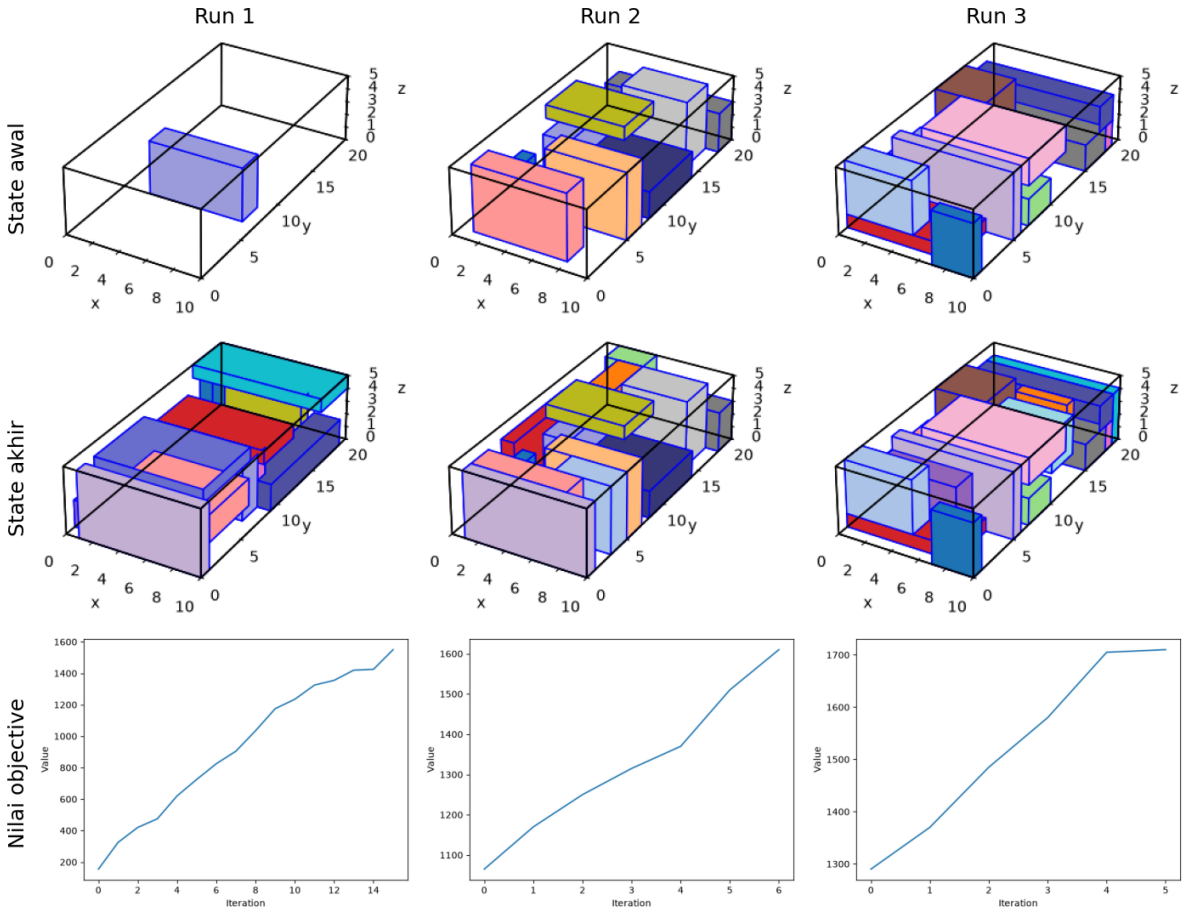
    while iterations < maxIter:
        improved = False
        for _ in range(trials):
            total_attempts += 1
            new_state = randomNeighbor(current, packages, truck)
            if new_state is None:
                continue
            new_value = stateScore(new_state, packages, objective)
            if new_value > current_value:
                current, current_value = new_state, new_value
                values.append(current_value)
                iterations += 1
                improved = True
                break
        if not improved:
            break

    return {
        "variant": "stochastic",
        "objective": objective,
        "seed": seed,
        "initial": initial,
        "final": current,
        "finalVal": current_value,
        "realValue": rawValue(current, packages),
        "duration": time.perf_counter() - start,
        "valuesOT": values,
        "iterations": iterations,
        "max_iter": maxIter,
        "trials_per_iteration": trials,
        "total_attempts": total_attempts
```

```
    }
```

Tabel 2. Hasil stochastic hill climbing (100 percobaan per iterasi).

Run (seed)	Nilai akhir	Durasi (detik)	Banyak iterasi
1	1550	0,296	15
2	1610	0,196	6
3	1710	0,276	5



Gambar 2. Stochastic hill climbing, run 1 sampai 3.

Hanya 5 sampai 15 langkah yang diterima sebelum pencarian berhenti. Nilai akhirnya paling rendah (1550–1710). Setelah neighbor lebih baik menipis, 100 percobaan random sering gagal.

Hill Climbing with Sideways Move

Seperti steepest ascent tapi jika tidak ada langkah yang menaikkan nilai, langkah bernilai sama bisa diambil. Parameter maxSideways membatasi jumlah penggunaan gerakan menyamping.

findSidewaysMove digunakan untuk mencari gerakan yang bisa dilakukan.

```
def findSidewaysMove(state, packages, truck, objective="value",
prev_key=None):
    scores = {k: packageScore(packages[k], objective) for k in packages}
    inside = sorted(k for k, placement in state.items() if
placement["position"] is not None)
    outside = sorted(k for k, placement in state.items() if
placement["position"] is None)
    current_key = geometryKey(state, packages)

    def acceptable(candidate):
        if candidate is None:
            return False
        key = geometryKey(candidate, packages)
        return key != current_key and (prev_key is None or key != prev_key)

    for a, b in itertools.combinations(inside, 2):
        candidate = trySwapPositions(state, a, b, packages, truck)
        if acceptable(candidate):
            return candidate, f"swap posisi {a} <-> {b}"

    for i in inside:
        for j in outside:
            if scores[j] != scores[i]:
                continue
            candidate = trySwapPositions(state, i, j, packages, truck)
            if acceptable(candidate):
                return candidate, f"swap posisi {i} <-> {j}"

    for k in inside:
        current_ori = state[k]["orientation"]
        current_dims = tuple(rotatedDims(k, current_ori, packages))
        for orientation in ORIENTATIONS:
            if orientation == current_ori:
                continue
            if tuple(rotatedDims(k, orientation, packages)) == current_dims:
```

```
                continue  # rotasi yang tidak mengubah dimensi efektif
            candidate = fotokopi(state)
            candidate[k] = {"position": state[k]["position"], "orientation":
orientation}
            if state_valid(candidate, packages, truck) and
acceptable(candidate):
                return candidate, f"rotasi paket {k} ke orientasi
{orientation}"

    for k in inside:
        current_pos = state[k]["position"]
        reduced = fotokopi(state)
        reduced[k] = {"position": None, "orientation":
state[k]["orientation"]}
        placement = findFeasiblePlacement(reduced, k, packages, truck,
exclude=current_pos)
        if placement is None:
            continue
        position, orientation = placement
        candidate = fotokopi(reduced)
        candidate[k] = {"position": position, "orientation": orientation}
        if state_valid(candidate, packages, truck) and acceptable(candidate):
            return candidate, f"pindah paket {k} ke {position}"

    return None, None
```

sidewaysMove mengambil langkah lebih baik kalau ada dan langkah sideways sebaliknya jika ada

```
def sidewaysMove(packages, truck, objective="value", maxSideways=100,
maxIter=2000, seed=None):
    if seed is not None:
        random.seed(seed)

    start = time.perf_counter()
    initial = gen_initial_state(packages, truck)
    current = fotokopi(initial)
    current_value = stateScore(current, packages, objective)

    values = [current_value]
    iterations = 0
    sideways_total = 0
    previous_key = None
```

```
    stopped_by = "no_move"

    while iterations < maxIter:
        best = bestImprovingMove(current, packages, truck, objective)
        if best is not None:
            _, new_state, _ = best
            previous_key = geometryKey(current, packages)
            current = new_state
            current_value = stateScore(current, packages, objective)
            values.append(current_value)
            iterations += 1
            continue

        if sideways_total >= maxSideways:
            stopped_by = "max_sideways"
            break

        new_state, _ = findSidewaysMove(current, packages, truck, objective,
previous_key)
        if new_state is None:
            stopped_by = "no_move"
            break

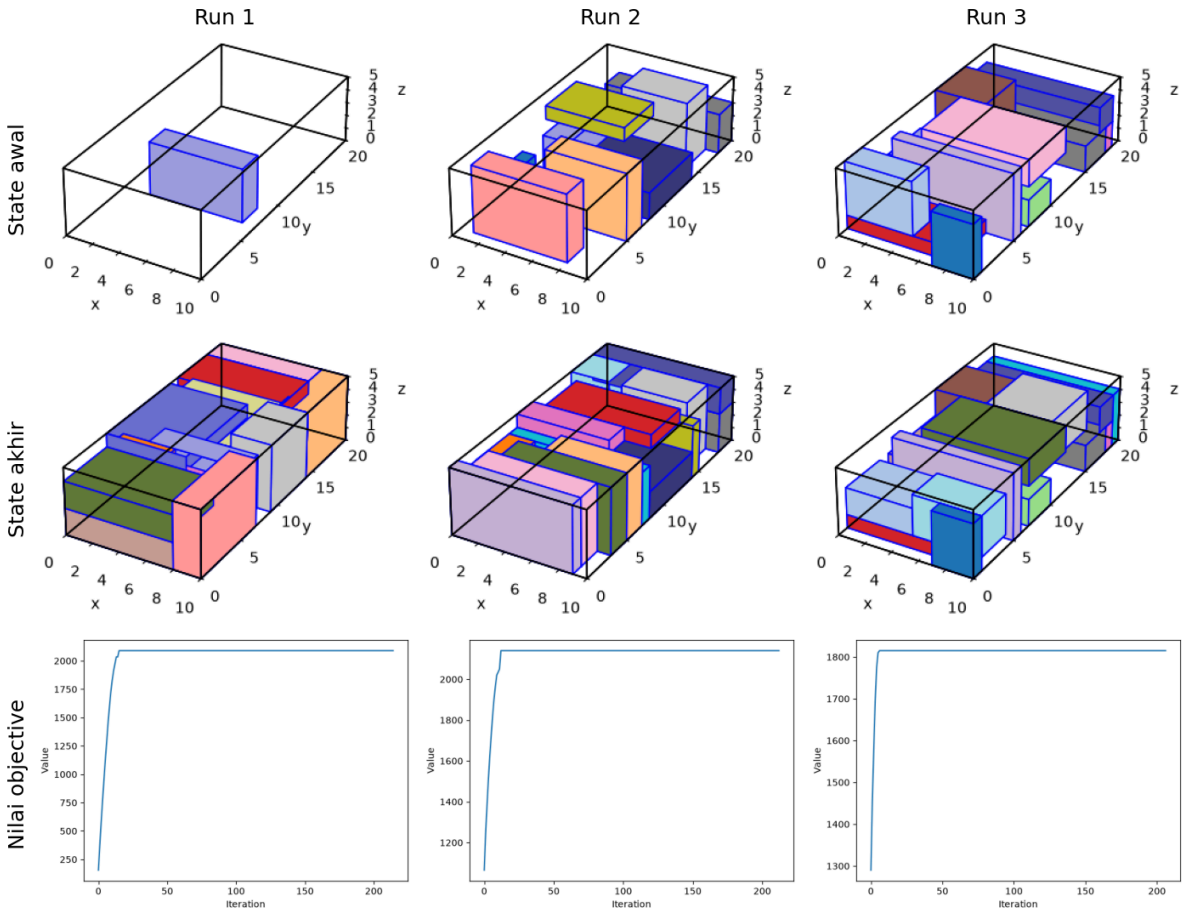
        previous_key = geometryKey(current, packages)
        current = new_state
        values.append(current_value)
        iterations += 1
        sideways_total += 1

    return {
        "variant": "sideways_move",
        "objective": objective,
        "seed": seed,
        "initial": initial,
        "final": current,
        "finalVal": current_value,
        "realValue": rawValue(current, packages),
        "duration": time.perf_counter() - start,
        "valuesOT": values,
        "iterations": iterations,
        "max_iter": maxIter,
        "max_sideways": maxSideways,
        "sideways_moves": sideways_total,
```

```
        "stopped_by": stopped_by
    }
```

Tabel 3. Hasil hill climbing with sideways move (maxSideways = 200).

Run (seed)	Nilai akhir	Durasi (detik)	Banyak iterasi	Sideways move
1	2090	6,030	214	200
2	2140	7,677	212	200
3	1815	6,108	206	200



Gambar 3. Hill climbing with sideways move, run 1 sampai 3.

Ketiga run berhenti karena batas 200 sideways tercapai. Hanya run 1 yang lebih baik dari steepest ascent (2035 → 2090), sedangkan run 2 dan 3 berakhir di nilai yang sama. Tipe HC ini paling lambat di antara yang lainnya (6 sampai 7,7 detik) karena memeriksa lebih banyak kandidat.

Random Restart Hill Climbing

Steepest ascent diulang dari initial state acak yang berbeda, dan solusi terbaiknya disimpan. Parameter maxRestart menghentikan jika jumlah restart mencapai batas yang ditentukan.

```
def randomRestart(packages, truck, objective="value", maxRestart=5,
maxIterPerRestart=2000, seed=None):
    if seed is not None:
        random.seed(seed)

    start = time.perf_counter()
    best_state = None
    best_value = None
    first_initial = None
    iterations_per_restart = []
    restart_values = []
    values = []
    restarts = 0

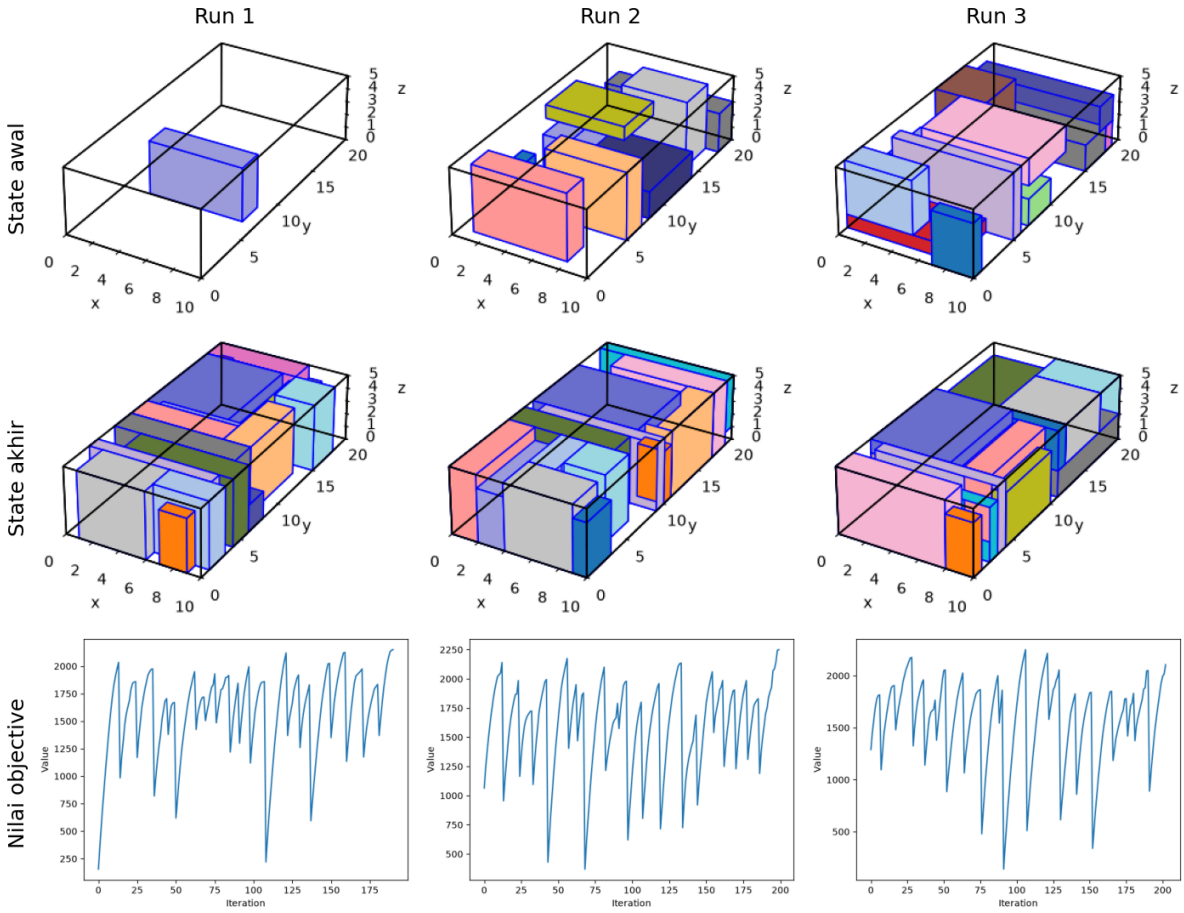
    while restarts < maxRestart:
        # seed=None: tiap restart mengambil initial state random baru dari RNG
global
        result = steepestAscent(packages, truck, objective,
maxIter=maxIterPerRestart, seed=None)
        if first_initial is None:
            first_initial = result["initial"]
        restarts += 1
        iterations_per_restart.append(result["iterations"])
        restart_values.append(result["finalVal"])
        values.extend(result["valuesOT"])
        if best_value is None or result["finalVal"] > best_value:
            best_value = result["finalVal"]
            best_state = result["final"]

    return {
        "variant": "random_restart",
        "objective": objective,
        "seed": seed,
        "initial": first_initial,
        "final": best_state,
        "finalVal": best_value,
        "realValue": rawValue(best_state, packages),
```

```
        "duration": time.perf_counter() - start,
        "valuesOT": values,
        "iterations": sum(iterations_per_restart),
        "max_iter": maxIterPerRestart,
        "max_restart": maxRestart,
        "restarts": restarts,
        "iterations_per_restart": iterations_per_restart,
        "restart_final_values": restart_values
    }
```

Tabel 4. Hasil random restart hill climbing (maxRestart = 20).

Run (seed)	Nilai akhir	Durasi (detik)	Banyak restart	Total iterasi	Iterasi per restart
1	2150	2,169	20	171	13, 10, 10, 8, 4, 12, 5, 6, 8, 5, 6, 9, 13, 7, 6, 12, 9, 10, 9, 9
2	2250	2,182	20	180	12, 10, 8, 9, 13, 5, 4, 13, 8, 5, 9, 11, 14, 9, 10, 6, 7, 7, 7, 13
3	2250	2,035	20	183	6, 9, 11, 7, 7, 6, 11, 11, 11, 2, 15, 14, 7, 10, 10, 13, 9, 4, 9, 11



Gambar 4. Random restart hill climbing, run 1 sampai 3 (plot mencakup seluruh restart).

Random restart memberi nilai tertinggi dan paling stabil (2150–2250). Setiap restart hanya butuh 2 sampai 15 iterasi, sehingga 20 restart tetap selesai sekitar 2 detik. Hasil menunjukkan mencoba banyak titik awal cukup untuk menemukan local optimum yang lebih baik.

Simulated Annealing

State menyimpan paket yang ada di truk. Langkah yang menurunkan nilai diterima dengan peluang exp(ΔE/T). Suhu awal 250 dikali 0,9999 setiap iterasi, dan pencarian kalau suhu mencapai 10,

acc menghitung peluang penerimaan.

```
def acc(delta, T):
    return math.exp( delta / T)
```

ranbor membangkitkan neighbor acak dari tiga langkah: swap, move, dan rotate. Hanya neighbor valid yang dikembalikan.

```
def ranbor(state, packages, truck, t=50):
    W, L, H = truck["dimensions"]
    aksis = {"x": (1, 2), "y": (0, 2), "z": (0, 1)}
    for _ in range(t):
        move = random.choice(["swap", "move", "rotate"] if state else
["move"])
        new = fotokopi(state)
        if move == "swap":
            a = random.choice(list(state))
            b = random.choice([id for id in packages if id != a])
            if b in state:
                new[a]['position'], new[b]['position'] = state[b]['position'],
state[a]['position']
            else:
                new[b] = state[a]
                del new[a]
        elif move == "move":
            id = random.choice(list(packages))
            if id in state and random.random() < 0.51:
                del new[id]
            else:
                orient = state[id]["orientation"] if id in state else
random.sample((0, 1, 2), 3)
                w, l, h = dims(id, orient, packages)
                new.pop(id, None)
                floor = [z for z in possible_height(new, packages) if z + h <=
H]
                if not floor or w > W or l > L:
                    continue
                pos = (random.randint(0, W - w), random.randint(0, L - l),
random.choice(floor))
                new[id] = {"position": pos, "orientation": orient}
        else:
            id = random.choice(list(state))
            i, j = aksis[random.choice("xyz")]
            orient = list(state[id]["orientation"])
            orient[i], orient[j] = orient[j], orient[i]
            new[id] = {"position": state[id]["position"], "orientation":
orient}
        if state_valid(new, packages, truck):
```

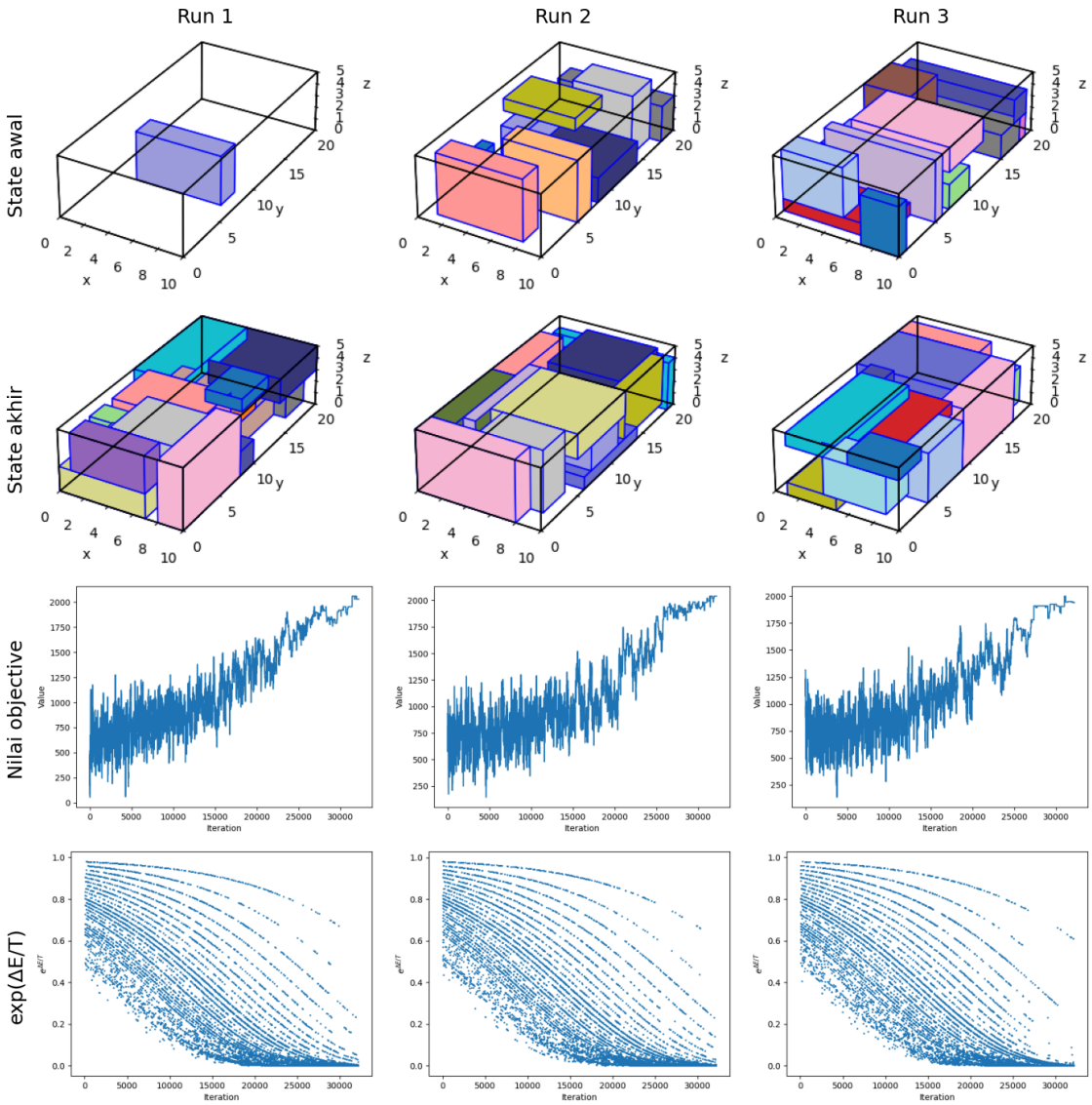
```
            return new
    return None
```

Run menjalankan fungsi utama. Stuck disini diartikan tidak ada perubahan nilai terbaik setelah 800 iterasi

```
def Run(packages, truck, TStart=250, cooling=0.9999, Tmin=10, t=40000,
seed=None):
    random.seed(seed)
    start = time.perf_counter()
    initial = cur = initState(packages, truck)
    curVal = truckValue(cur, packages)
    T = TStart
    values, probs = [], []
    maxVal, noInc, stuck = curVal, 0,0
    for _ in range(t):
        if T < Tmin:
            break
        new = ranbor(cur, packages, truck)
        prob = None
        if new is not None:
            delta = truckValue(new, packages) - curVal
            prob = acc(delta,T) if delta < 0 else None
            if delta >= 0 or random.random() < prob:
                cur, curVal = new, curVal + delta
        values.append(curVal)
        probs.append(prob)
        T *= cooling
        if curVal> maxVal:
            maxVal, noInc = curVal,0
        else:
            noInc += 1
            if noInc % (t // 50) == 0: stuck +=1
    return {
        "initial": initial, "final": cur, "finalVal": curVal,
        "duration": time.perf_counter() - start,
        "valuesOT": values, "probs": probs, "stuck": stuck
    }
```

Tabel 5. Hasil simulated annealing (T0 = 250, pendinginan 0,9999, Tmin = 10).

Run (seed)	Nilai akhir	Durasi (detik)	Frekuensi stuck
1	2030	3,302	33
2	2040	2,866	34
3	1940	3,071	35



Gambar 5. Simulated annealing, run 1 sampai 3.

Nilai akhir SA (1940–2040) stabil dan berada di antara stochastic dan random restart. Plot exp(ΔE/T) menunjukkan peluang menerima langkah yang memburuk mendekati 1

di awal lalu turun ke 0 seiring suhu turun. Frekuensi stuck 33 sampai 35 berarti lebih dari 26.000 dari sekitar 32.000 iterasi tidak menghasilkan nilai terbaik baru.

Genetic Algorithm

Setiap individu memiliki lima gen, [a, x, y, z, orientasi] dengan a = 1 jika paket ada di dalam truk. Tiap generasi simpan individu terbaik, pilih parent dengan *roulette wheel*, crossover satu titik, dan mutasi anak.

encode_state dan decode_state mengubah state jadi kromosom dan sebaliknya.

```
# Mengubah state menjadi array satu dimensi
def encode_state(state, packages):
    chromosome = []

    for package_id in packages:
        placement = state[package_id]
        position = placement["position"]
        orientation = placement["orientation"]

        ori_id = ORIENTATIONS.index(orientation)
        if position is None:
            chromosome.extend([0, 0, 0, 0, ori_id])

        else:
            chromosome.extend([
                1,
                position[0],
                position[1],
                position[2],
                ori_id]
            )

    return chromosome

def decode_state(chromosome, packages):
    state = {}

    for i, package_id in enumerate(packages):
        a, x, y, z, ori_id = chromosome[(i*5):(i*5+5)]

        if a == 0:
            position = None
```

```
        elif a == 1:
            position = (x, y, z)
        else:
            raise ValueError("Nilai kondisi posisi tidak boleh lebih dari 1")
#

        orientation = ORIENTATIONS[ori_id]
        state[package_id] = {
            "position" : position,
            "orientation" : orientation
        }

    return state
```

gen_initial_pop bangkitkan populasi awal dengan gen_initial_state. Satu kromosom diambil acak.

```
# Inisialisasi populasi
def gen_initial_pop(packages, truck, pop_size):
    population = []

    for _ in range(pop_size):
        state = gen_initial_state(packages, truck)
        chromosome = encode_state(state, packages)

        population.append(chromosome)

    inital_chromosome_random = random.choice(population)

    return inital_chromosome_random, population
```

fitness adalah total value paket di dalam truk.

```
# Menghitung fitness suatu chromosome
def fitness(chromosome, packages, truck):
    state = decode_state(chromosome, packages)

    if not state_valid(state, packages, truck):
        return float("-inf") #

    total_value = 0

    for package_id, placement in state.items():
```

```
        if placement["position"] is not None:
            total_value += packages[package_id]["value"]

    return total_value
```

roulette_selection memilih parent dengan peluang sebanding fitness.

```
def roulette_selection(population, packages, truck):
    fitness_value = [
        fitness(chromosome, packages, truck)
        for chromosome in population
    ]

    total_fitness = sum(fitness_value)
    if total_fitness == 0:
        return random.choice(population).copy()
    random_number = random.random() * 100

    cum_probability = 0

    for i in range(len(population)):
        probability = (fitness_value[i] / total_fitness) * 100

        cum_probability += probability

        # Memilih parent
        if random_number < cum_probability:
            return population[i].copy()

    return population[-1].copy()
```

crossover memotong kedua parent di batas antar paket, sehingga gen satu paket tidak pernah terbelah.

```
def crossover(parent_1, parent_2, packages):
    num_packages = len(packages)

    if (num_packages < 2):
        return parent_1.copy()
    cut_index = (random.randint(1, num_packages - 1)) * 5
    child = parent_1[:cut_index] + parent_2[cut_index:]
    return child
```

mutation ubah satu paket secara acak.

```
def mutation(chromosome, packages, truck):
    mutated = chromosome.copy()

    if not packages:
        return mutated
    package_idx = random.randint(0, len(packages) - 1)
    package_id = list(packages.keys())[package_idx]

    start = package_idx * 5
    a, x, y, z, ori_id = mutated[start:start+5]
    dimensions = packages[package_id]["dimensions"]
    truck_dims = truck["dimensions"]
    orientation = ORIENTATIONS[ori_id]
    rotate_dims = [dimensions[i] for i in orientation]

    if a == 0:
        operation = "add"
    else:
        operation = random.choice(["remove", "move", "rotate"])

    if operation == "add":
        ori_id = random.randint(0, 5)
        orientation = ORIENTATIONS[ori_id]
        rotate_dims = [dimensions[i] for i in orientation]

        if any(rotate_dims[k] > truck_dims[k] for k in range(3)):
            return mutated

        state = decode_state(chromosome, packages)

        heights = [
            h for h in possible_height(state, packages)
            if h + rotate_dims[2] <= truck_dims[2]
        ]

        if not heights:
            return mutated

        position = (
            random.randint(0, truck_dims[0] - rotate_dims[0]),
            random.randint(0, truck_dims[1] - rotate_dims[1]),
```

```
            random.choice(heights)
        )

        mutated[start:start+5] = [
            1,
            position[0],
            position[1],
            position[2],
            ori_id
        ]

    elif operation == "remove":
        mutated[start:start+5] = [0, 0, 0, 0, ori_id]

    elif operation == "move":
        position = [x, y, z]
        axis = random.randint(0, 2)

        if (axis == 0) or (axis == 1):
            choices = range(truck_dims[axis] - rotate_dims[axis] + 1)
        else:
            state = decode_state(chromosome, packages)
            state[package_id]["position"] = None

            choices = [
                h for h in possible_height(state, packages)
                if h + rotate_dims[2] <= truck_dims[2]
            ]

        choices = [value for value in choices if value != position[axis]]

        if not choices:
            return mutated
        position[axis] = random.choice(choices)
        mutated[start+1:start+4] = position

    elif operation == "rotate":
        opt_rotate = [i for i in range(6) if i != ori_id]
        mutated[start+4] = random.choice(opt_rotate)

    return mutated
```

Hasil crossover dan mutasi bisa tabrakan atau melayang. Repair_state memperbaikinya. Paket yang tidak valid dicoba pada posisi di dinding atau paket lain, dan dikeluarkan kalau tidak ada solusi.

```
def repair_state(state, packages, truck):
    repaired = {}
    for package_id in packages:
        repaired[package_id] = {
            "position" : None,
            "orientation" : state[package_id]["orientation"]
        }
    list_package_id = [
        package_id for package_id in packages
        if state[package_id]["position"] is not None
    ]

    list_package_id.sort(key=lambda package_id: (
        state[package_id]["position"][2],
        -packages[package_id]["value"]))

    truck_dims = truck["dimensions"]

    for package_id in list_package_id:
        original_position = state[package_id]["position"]
        orientation = state[package_id]["orientation"]
        dimensions = packages[package_id]["dimensions"]
        rotate_dims = [dimensions[i] for i in orientation]

        repaired[package_id]["position"] = original_position
        if state_valid(repaired, packages, truck):
            continue

        repaired[package_id]["position"] = None

        if any(rotate_dims[k] > truck_dims[k] for k in range(3)):
            continue
        heights = [
            h for h in possible_height(repaired, packages)
            if h + rotate_dims[2] <= truck_dims[2]
        ]

        # Mengambil X dan Y pilihan
```

```
        x_choices = {0, original_position[0]}
        y_choices = {0, original_position[1]}

        # Menambahkan pilihan berdasarkan posisi paket lain
        for other_id in packages:
            other_position = repaired[other_id]["position"]

            if other_position is None:
                continue

            other_orientation = repaired[other_id]["orientation"]
            other_dimensions = packages[other_id]["dimensions"]
            other_rotate_dims = [
                other_dimensions[i] for i in other_orientation
            ]

            ox, oy, oz = other_position
            ow, ol, oh = other_rotate_dims

            x_choices.update([
                ox,
                ox + ow,
                ox - rotate_dims[0],
                ox + ow - rotate_dims[0]
            ])

            y_choices.update([
                oy,
                oy + ol,
                oy - rotate_dims[1],
                oy + ol - rotate_dims[1]
            ])

        # Jika pilihan posisi masih di dalam truk
        x_choices = [
            x for x in x_choices
            if 0 <= x <= truck_dims[0] - rotate_dims[0]
        ]

        y_choices = [
            y for y in y_choices
            if 0 <= y <= truck_dims[1] - rotate_dims[1]
        ]
```

```
        # pilihan yang sudah difilter X dan Y
        positions_xy = [
            (x, y)
            for x in x_choices
            for y in y_choices
        ]

        random.shuffle(positions_xy)

        # cari posisi valid
        found = False
        for z in sorted(heights):
            for x, y in positions_xy:

                repaired[package_id]["position"] = (x, y, z)
                if state_valid(repaired, packages, truck):
                    found = True
                    break
            if found:
                break
        if not found:
            repaired[package_id]["position"] = None
    return repaired
```

repair_chromosome menerapkan repair_state pada kromosom.

```
def repair_chromosome(chromosome, packages, truck):

    state = decode_state(chromosome, packages)
    repaired_state = repair_state(state, packages, truck)
    return encode_state(repaired_state, packages)
```

raw_value menghitung total value tanpa memeriksa constraint, untuk membandingkan value sebelum dan sesudah repair.

```
# Menghitung value yang hilang saat di repair
def raw_value(chromosome, packages):
    state = decode_state(chromosome, packages)
    total = 0
    for package_id in packages:
        if state[package_id]["position"] is not None:
            total += packages[package_id]["value"]
```

```
    return total
```

genetic_algorithm menjalankan fungsi fungsi utama dan mencatat fitness maksimum dan rata-rata fitness populasi tiap iterasi.

```
def genetic_algorithm(packages, truck, pop_size, max_iter, mutation_rate=0.1):
    if pop_size < 1 or max_iter < 0:
        raise ValueError("Jumlah populasi dan maksimal iterasi tidak valid")
    if not 0 <= mutation_rate <= 1:
        raise ValueError("Mutation rate tidak valid")
    initial_chromosome_random, population = gen_initial_pop(packages, truck,
pop_size)

    fitness_values = [
        fitness(chromosome, packages, truck)
        for chromosome in population
    ]

    best_idx = fitness_values.index(max(fitness_values))
    best_chromosome = population[best_idx].copy()
    best_fitness = fitness_values[best_idx]

    history_best = [best_fitness]
    history_curr_best = [best_fitness]
    history_avg = [sum(fitness_values) / pop_size]


    for _ in range(max_iter):
        new_population = [best_chromosome.copy()]
        while len(new_population) < pop_size:

            parent_1 = roulette_selection(population, packages, truck)
            parent_2 = roulette_selection(population, packages, truck)

            child = crossover(parent_1, parent_2, packages)

            value_before = raw_value(child, packages)
            child = repair_chromosome(child, packages, truck)
            value_after = raw_value(child, packages)

            if random.random() < mutation_rate:
                child = mutation(child, packages, truck)
            child = repair_chromosome(child, packages, truck)
```

```
            new_population.append(child) # Memasukkan child ke populasi baru

        population = new_population
        fitness_values = [
            fitness(chromosome, packages, truck)
            for chromosome in population
        ]

        curr_best_idx = fitness_values.index(max(fitness_values))
        curr_best_fitness = fitness_values[curr_best_idx]

        if curr_best_fitness > best_fitness:
            best_fitness = curr_best_fitness
            best_chromosome = population[curr_best_idx].copy()

        history_best.append(best_fitness)
        history_curr_best.append(curr_best_fitness)
        history_avg.append(sum(fitness_values) / pop_size)

    return initial_chromosome_random, best_chromosome, history_best,
history_curr_best, history_avg
```

Eksperimen memakai mutation rate 0,1. Jumlah populasi 30 di tes dengan banyak iterasi 25, 50, dan 100 lalu banyak iterasi 50 di tes dengan jumlah populasi 10, 30, dan 50. Setiap konfigurasi dijalankan tiga kali.

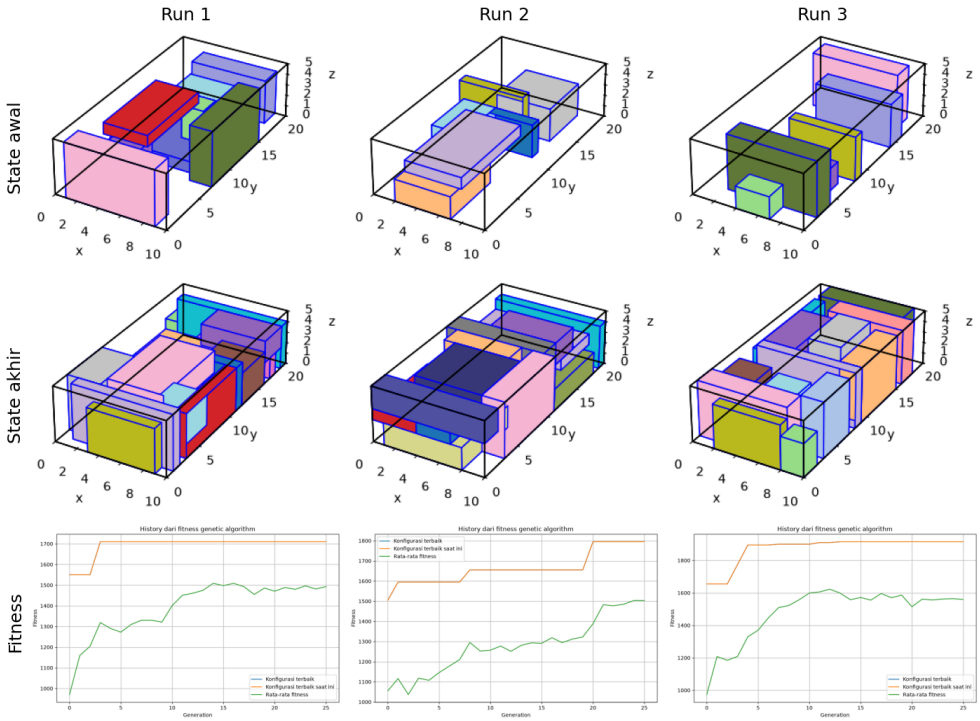
Tabel 6. Hasil genetic algorithm untuk setiap konfigurasi.

Jumlah populasi	Banyak iterasi	Run	Nilai akhir	Durasi (detik)
30	25	1	1710	6,135
30	25	2	1795	4,618
30	25	3	1915	5,245
30	50	1	1995	10,297
30	50	2	1950	13,820
30	50	3	1915	10,066
30	100	1	2005	30,554

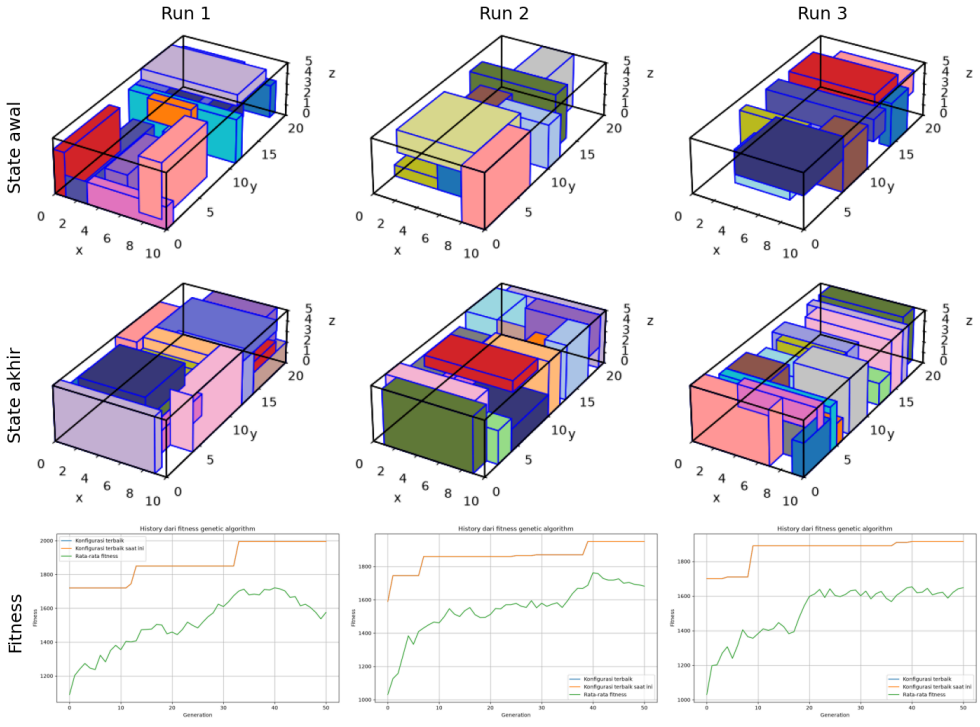
Jumlah populasi	Banyak iterasi	Run	Nilai akhir	Durasi (detik)
30	100	2	1730	21,346
30	100	3	1990	24,114
10	50	1	1655	1,230
10	50	2	1755	1,656
10	50	3	1610	1,443
50	50	1	1935	21,929
50	50	2	1910	21,985
50	50	3	2045	30,476

Tabel 7. Ringkasan genetic algorithm per konfigurasi (tiga run).

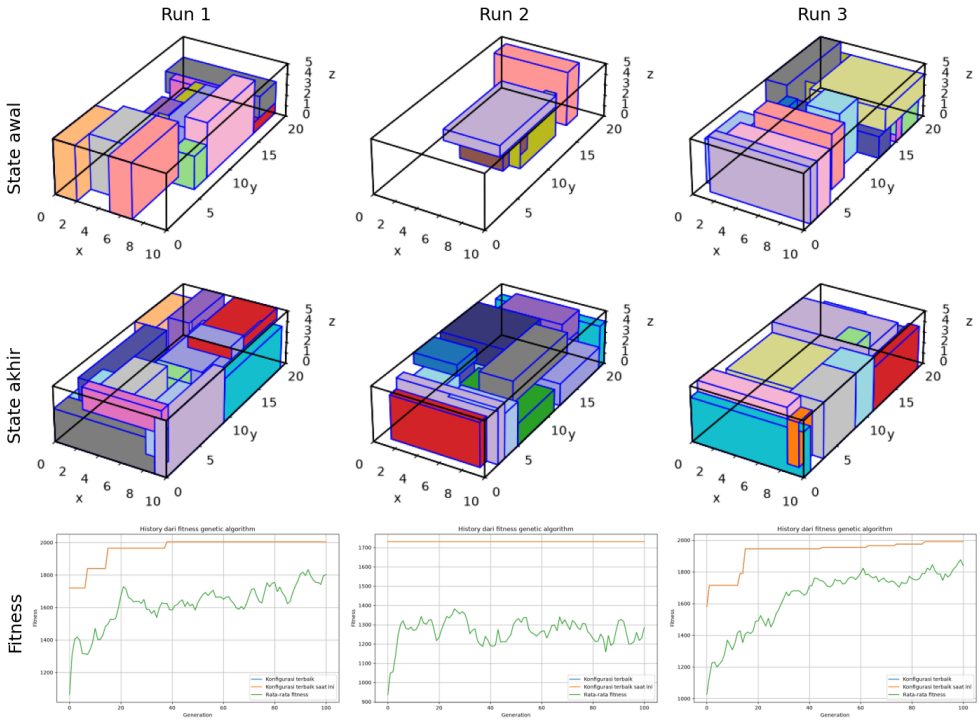
Jumlah populasi	Banyak iterasi	Rata-rata nilai	Rentang nilai	Durasi rata-rata (detik)
30	25	1806,7	1710–1915	5,333
30	50	1953,3	1915–1995	11,394
30	100	1908,3	1730–2005	25,338
10	50	1673,3	1610–1755	1,443
50	50	1963,3	1910–2045	24,797



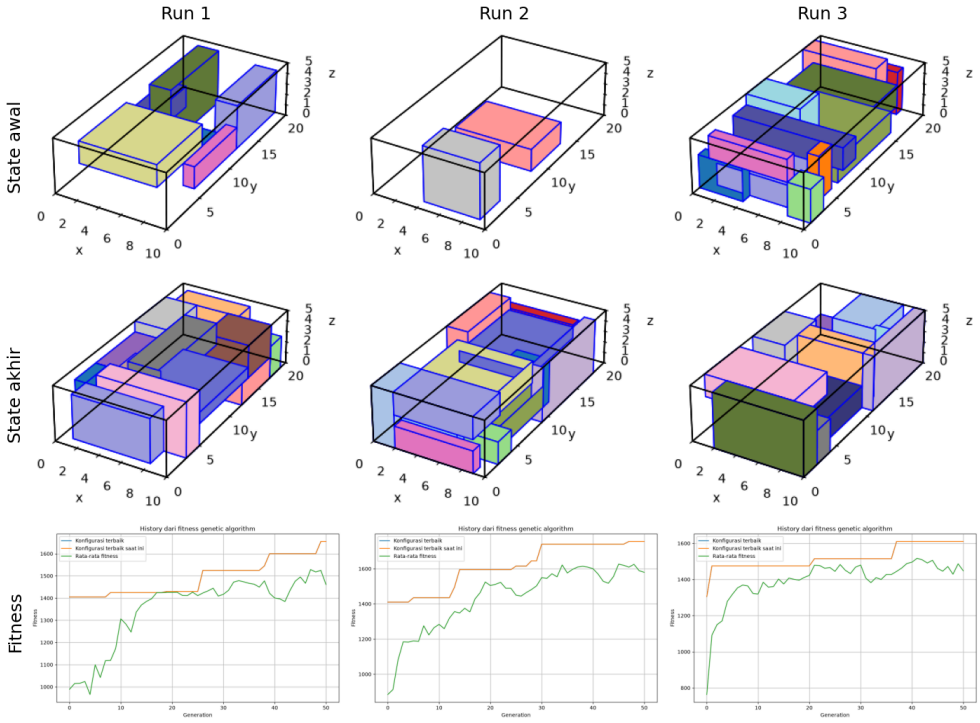
Gambar 6. Genetic algorithm dengan populasi 30 dan 25 iterasi.



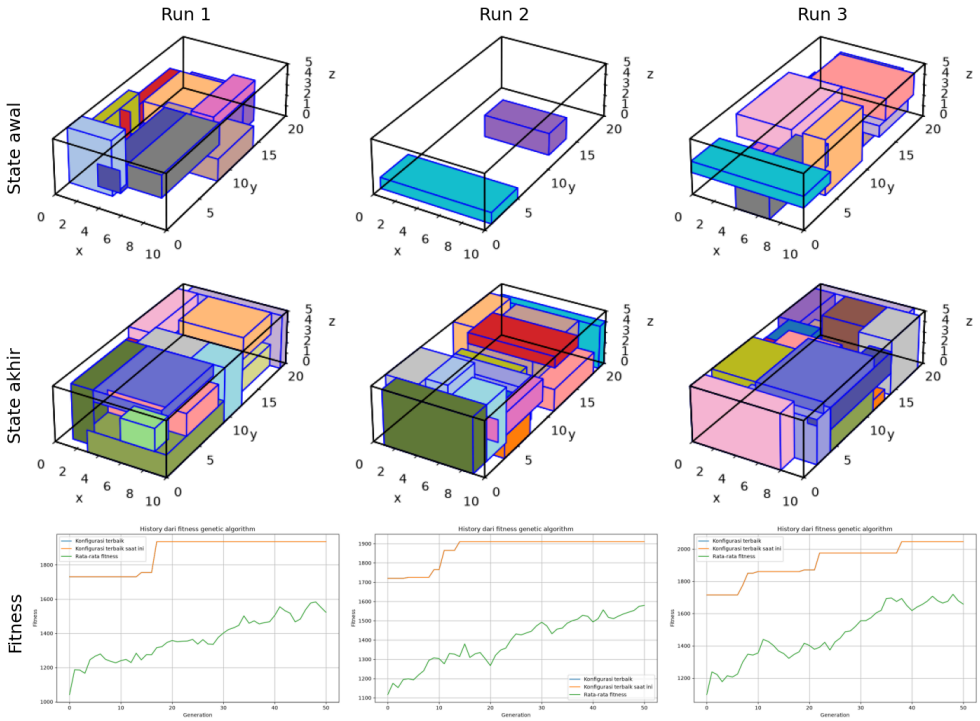
Gambar 7. Genetic algorithm dengan populasi 30 dan 50 iterasi.



Gambar 8. Genetic algorithm dengan populasi 30 dan 100 iterasi.



Gambar 9. Genetic algorithm dengan populasi 10 dan 50 iterasi.



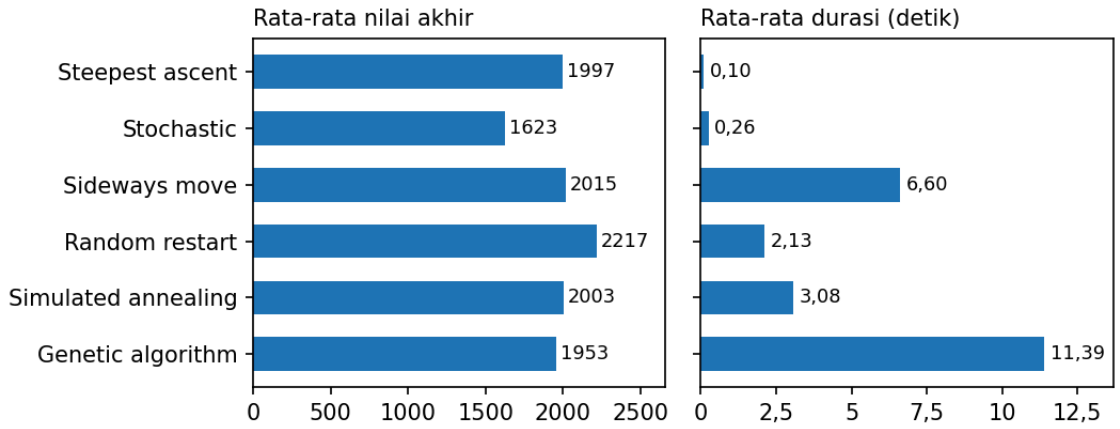
Gambar 10. Genetic algorithm dengan populasi 50 dan 50 iterasi.

Karena elitisme, fitness maksimum tidak pernah turun, sedangkan rata-rata fitness populasi naik lebih lambat dan tetap di bawahnya.

Analisis Hasil Eksperimen

Tabel 8. Perbandingan seluruh algoritma (tiga run; genetic algorithm dengan populasi 30 dan 50 iterasi).

Algoritma	Nilai terbaik	Rata-rata nilai	Durasi rata-rata (detik)	Rentang nilai
Steepest ascent	2140	1996,7	0,099	1815–2140
Stochastic	1710	1623,3	0,256	1550–1710
Sideways move	2140	2015,0	6,605	1815–2140
Random restart	2250	2216,7	2,129	2150–2250
Simulated annealing	2040	2003,3	3,080	1940–2040
Genetic algorithm	1995	1953,3	11,394	1915–1995



Gambar 11. Rata-rata nilai akhir dan durasi setiap algoritma.

Seberapa dekat tiap-tiap algoritma mendekati global optima

Kita tidak mengetahui global optima tapi melalui perbandingan antara tiap algoritma nilai terbaik dicapai oleh random restart lalu steepest ascent, sideways move, genetic algorithm, simulated annealing, dan stochastic. Random restart paling dekat karena mencoba 20 titik awal. Stochastic paling buruk karena akan stop begitu saja dengan 100 percobaan gagal. Simulater Annealing hanya bisa keluar dari local optimum selama suhu masih tinggi, dan Genetic Algorithm bergantung pada populasi awal.

Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain?

Random restart (2216,7), sideways move (2015,0), simulated annealing (2003,3), steepest ascent (1996,7), genetic algorithm (1953,3), lalu stochastic (1623,3). Sideways move hanya sekali lebih baik dari steepest ascent, jadi selisihnya kecil. SA dan GA bisa keluar dari local optimum tapi dengan parameter yang dipakai belum melewati steepest ascent.

Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?

Steepest ascent (0,099 detik) dan stochastic (0,256 detik) paling cepat karena berhenti di local optimum pertama. Random restart (2,129 detik) sekitar 20 kali steepest ascent karena menjalankan 20 pencarian. SA (3,080 detik) menjalankan sekitar 32.000 iterasi yang murah. Sideways move (6,605 detik) lebih lambat karena memeriksa banyak kandidat. GA paling lama (11,394 detik pada populasi 30 dan 50 iterasi) karena setiap iterasi membuat dan memperbaiki berbagai anak.

Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?

Steepest ascent dan sideways move paling tidak konsisten karena bergantung dengan initial state. Random restart konsisten karena ambil yang terbaik dari 20 initial state. Pada GA, rentangnya 80 sampai 275 tergantung konfigurasi (Tabel 7), karena populasi awal selalu acak.

Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?

Dengan populasi 30, tambahan iterasi dari 25 ke 50 menaikkan rata-rata dari 1806,7 ke 1953,3. Pada 100 iterasi, dua run mencapai 2005 dan 1990. Jadi jumlah iterasi meningkatkan kemungkinan ditemukannya anak yang lebih baik

Dengan 50 iterasi, menaikkan populasi dari 10 ke 30 menaikkan rata-rata dari 1673,3 ke 1953,3, sedangkan populasi 50 hanya sedikit lebih tinggi (1963,3) dengan durasi lebih dari dua kali lipat. Populasi yang lebih besar memberi lebih banyak variasi untuk crossover, tetapi manfaatnya mengecil (*diminishing returns*) sehingga pada suatu saat penambahan populasi hanya akan memberi efek minimal dan bersifat negatif karena akan memperbersar beban komputasi.

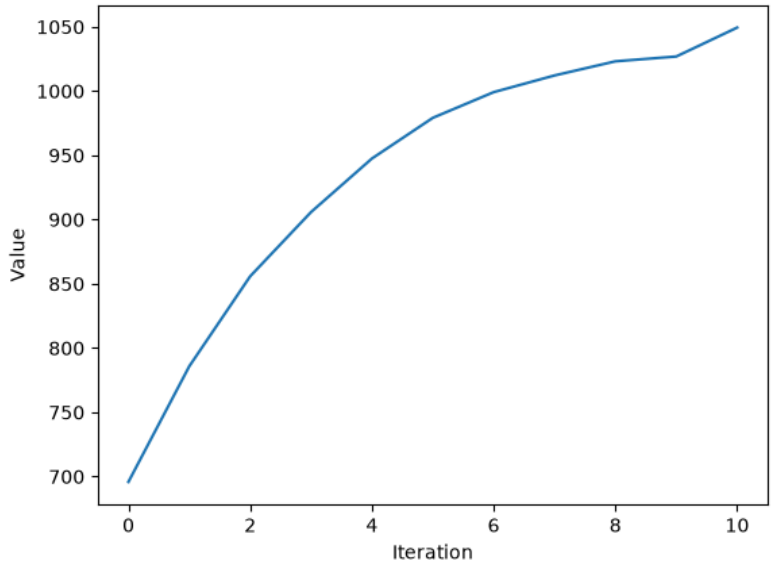
Durasi naik mendekati linear dengan banyak iterasi (5,3; 11,4; dan 25,3 detik) dan lebih cepat dari linear dengan jumlah populasi (1,4; 11,4; dan 24,8 detik).

Bonus: Objective Alternatif (Urgency)

Objective urgency memberi skor value/eta, sehingga paket dengan ETA kecil diprioritaskan. Hanya packageScore yang berubah, lalu steepest ascent dijalankan ulang dengan seed yang sama.

Tabel 9. Steepest ascent dengan objective value dan urgency.

Run (seed)	Nilai akhir (objective value)	Nilai akhir (objective urgency)	Skor urgency
1	2035	1975	1319,42
2	2140	2010	1049,42
3	1815	1975	1096,92



Gambar 12. Nilai objective urgency terhadap iterasi, steepest ascent run 2.

III. Kesimpulan dan Saran

Implementasi local search untuk permasalahan pengisian truk sudah selesai dan seluruh state yang dihasilkan pada eksperimen memenuhi *constraint*. Terdapat empat tipe Hill Climbing, Simulated Annealing, genetic algorithm, visualisasi state tiga dimensi, serta plot nilai objective dan probabilitas penerimaan simulated annealing.

Random restart hill climbing mencapai hasil terbaik (2250) dan paling konsisten (2150–2250), diikuti steepest ascent dan sideways move (maksimum 2140), simulated annealing (2040), dan stochastic hill climbing (1710).

Saran:

- Hitung fitness populasi sekali per iterasi, bukan setiap kali memilih parent, agar genetic algorithm dengan populasi besar lebih cepat.
- Gunakan crossover yang mempertahankan susunan paket, misalnya menukar satu wilayah truk, agar anak tidak banyak melanggar constraint.
- Uji dengan jumlah paket dan ukuran truk yang berbeda untuk melihat skalabilitas.

IV. Pembagian Tugas

Pembagian tugas antar anggota kelompok adalah sebagai berikut:

Tabel 10. Pembagian tugas anggota kelompok.

Anggota	NIM	Kontribusi
Mohammad Najmutstsaqib	13223040	Inisialisasi pemodelan 3D packing dan generator initial state, validasi constraint dasar, dan Genetic Algorithm.
Ega Luthfi Rais	13524115	Implementasi empat varian Hill Climbing dan heuristik penempatan paket, eksekusi dan visualisasi eksperimen Hill Climbing (tiga run per varian), bonus objective alternatif (urgency), Visualizer web based.
Ahmad Fauzan Putra	13524141	Implementasi Simulated Annealing, fungsi visualisasi dasar (plotState dan plotValues) dan laporan.

V. Referensi

[1] M. L. Khodra, “Beyond Classical Search: Classical vs Local Search,” YouTube, 2021. [Video]. Diakses: 16 September 2026.

[2] M. L. Khodra, “Beyond Classical Search: State (Value, Neighbor),” YouTube, 2021. [Video]. https://www.youtube.com/watch?v=_6rla37QorM. Diakses: 16 September 2026.

[3] M. L. Khodra, “Beyond Classical Search: Hill-climbing Search,” YouTube, 2021. [Video]. https://www.youtube.com/watch?v=L0b0CN3Fds8. Diakses: 16 September 2026.

[4] M. L. Khodra, “Beyond Classical Search: Simulated Annealing,” YouTube, 2021. [Video]. Diakses: 16 September 2026.

[5] M. L. Khodra, “Beyond Classical Search: Genetic Algorithm,” YouTube, 2021. [Video]. Diakses: 16 September 2026.

[6] Python Software Foundation, “The Python Tutorial,” https://docs.python.org/3/tutorial/. Diakses: 19 Oktober 2026.

[7] Python Software Foundation, “pathlib — Object-oriented filesystem paths,” https://docs.python.org/3/library/pathlib.html. Diakses: 19 Oktober 2026.

[8] Matplotlib Development Team, “mplot3d toolkit,” https://matplotlib.org/stable/api/toolkits/mplot3d.html. Diakses: 19 Oktober 2026.

VI. Lampiran Form Penggunaan Generative AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:
Nama/NIM :Mohammad Najmutstsaqib 13223040; Ega Luthfi Rais 13524115; Ahmad Fauzan Putra 13524141
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika (STEI)
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Inteligensi Artifisial
Judul Tugas/Proposal : Tugas Besar 1 - Searching

Menyatakan bahwa kami tidak menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

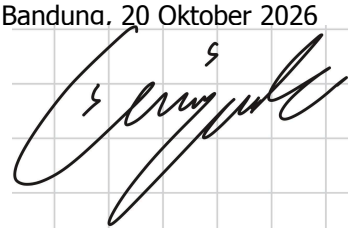
Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	NO AI
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	NO AI
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Tidak	NO AI
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	NO AI
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata

kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Mohammad Najmutstsaqib

13223040

Bandung, 20 Oktober 2026



Ega Luthfi Rais

13524115

Bandung, 20 Oktober 2026



Ahmad Fauzan Putra

13524141